数据结构重点

线性表

数组

- 如果Vector有点难理解，那么对于普通长度的数组一定要掌握
  - 其实Vector，就是做了两个事情。1.底层的数组使用的是地上的空间。但是它的操作和数组基本一致。 arr[1] 都可以使用 2.数组一旦装不下，这时候可以扩容。 堆空间申请的内存，可以使用realloc扩容
  - 建议大家代码一定要自己写一下。如果不写代码，这个过程纯靠背那不住
  - 过程一定要知道，且能用自己的话描述出来！
- 往数组指定下标插入数据 —— p[index]，将index往后的数据，整体往后挪一位。然后将要插入的数据放在该下标
- 删除指定下标数据
- 修改指定下标数据
- 查找数据是否存在

链表

- 头插法插入数据、尾插法插入数据
- 往指定下标插入数据数据，能掌握更好
  - 一般面试的问法，就是，简单描述一下链表插入数据的流程
  - 在链表上插入一个数据，需要找到它的前驱结点
  - 创建新结点，把数据存进去。
  - 新结点的指针域，指向前驱结点的指针域。
  - 前驱结点的指针域，再指向这个新结点
- 删除一个链表上的结点，掌握最基本的。比如开天窗了，给了你要删除数据的前一个结点，之后怎么删除下一个结点
  - 在面试的时候，回答大致就可以，比如使用临时指针存储一下待删除的。然后将前一个指针的next域，存储待删除的next域 最后将待删除的结点释放就可以了。 不要纯背代码
  - 不过自己写代码的时候，按照索引删建议掌握。因为可以锻炼自己的思维缜密性，写多了这种比较搞笑的代码，bug会少。
  - 删除指定索引的数据
  - 删除指定数据
- 改某个索引的数据。这个比较简单
- 查找某个数据，查找某个索引的数据

关于这种，面试官喜欢对照问。

- 比如，现在假设，这里有个数组，有个链表。 将它们的插入流程简单对比一下
- 比如删除流程
- 一定要注意的是，能够用自己的话说出来流程
- 简单介绍一下线性表（减分项）

线性表，就是一个线性的数据结构。 数据是一个接着一个的。 在物理上，有两种组织方式， 数组和链表。 数组的内存是连续的。链表的内存是离散的，主要是靠指针来指示下一个结点。
数组的查找是比较快的，插入和删除速度是比较慢的。 数组是有随机访问特点的，简单说，就是要访问第n个数据， 可以根据直接根据一个公式计算出来地址，然后去访问该地址。 base_addr+index*sizeof(int)
链表的插入，删除比较快。 查找是比较慢。

栈&队列

- 要知道基础的概念：什么是栈，什么是队列
- 栈是操作受限的线性表，只允许从一端进出
  - 数据后进先出
  - 栈用来存储函数调用过程
- 队列是 只允许从一端进，从另外一端出
  - 操作受限的线性表，只允许从一端进，从另一端出
  - 数据是先进先出
  - 可以保证公平。先来的先出，可以保证数据先到的先处理
- 怎么实现的
  - 使用数组
  - 使用链表
- 栈的实现
  - 一个普通的数组，再额外加上一个int类型的变量 来存储栈顶的位置， 进栈的时候，将数据放进去，再将这个int类型的变量 +1
  - 栈使用链表实现，是类似的
- 队列的实现
  - 线性表
  - 使用链表来实现队列。 就是从尾进数据，尾插。从头出数据。
  - 使用数组来实现。 循环队列的实现思路。
  - 什么是循环队列
    - 循环队列其实就是用数组的方式来实现队列。 它就是用两个下标来标识开始和结束的位置。在进出数据数据的时候，只需要挪动两个下标，不需要挪动数据。 这种做法，如果光+1.这时候会出现假满问题。 这时候我们采用的手段是，将 下标+1后对长度取余。 这时候如果超过了length。 这时候会从index=0继续走。 这就是循环的概念。 rear = (rear+1)%length; front = (front + 1) % length
    - 1.现在，使用什么下标来标识数据。
    - 一种方案是： 用一个int。每一次都需要挪数据
    - 第二种： 两个int. 出数据的时候，++int
    - 第三种：两个int。 tail = (tail +1)%len; 循环数组实现队列。
  - 循环队列的判空判满问题。
    - size
    - 如果满了，扩容，循环队列能不能用realloc。不能 应该用 malloc /calloc，然后将原始的数据拷贝过来。 从 front---->rear 最后，原始的空间需要释放掉。

树

- 要知道，什么是树

```
struct node_s{
int data;
struct node_s *left;
struct node_s *right;
}
```

数据的关系是一对多。 一个数据后面可以跟多个数据。
二叉树，三叉树

- 要知道二叉搜索树是什么
  - 二叉树：一个结点最多对两个孩子，就是二叉树
  - 孩子的大小关系和当前节点的关系是已知的。 左孩子比根节点小，右孩子比根节点大
- 要知道二叉搜索树的优点在哪
  - 查询的性能高。 但是是有前提的，这棵树应该不歪。
  - 如果数据量在100w。 不借助其他的数据结构，需要全部比一遍，性能很差。 如果借助二叉搜索树， 只需要一个20层的二叉搜索树就可以将数据全部存下来，换句话说，比较的时候， 只需要比20多次，就可以确定，是否有这个数据，其他的信息是什么。 100w 20次
- 红黑树是什么东西 —— 就是一个特殊的二叉搜索树，有一些自己的平衡策略。 能让这个树不歪
- 二叉树的实现方式有几种
  - 一个结点里面，有左右两个指针来实现的。
  - 和单链表很像。所以，操作很链表非常相似。
  - 数组也可以模拟二叉树
    - 一般数组模拟，要么是满二叉树，要么是完全二叉树。
    - 完全二叉树： 就是前面n-1层都是满的，最后一层，所有的结点都放在最左侧。
    - 满二叉树： 每一层都是满的
    - 这样的话，就可以直接按照数组的存储方式存储。 左孩子下标是 2i+1 右孩子是2i + 2 父节点的索引是i
  - 如果代码写不出来，基本的东西要知道
    - 什么是树，什么是二叉搜索树，什么是红黑树（作用是什么）。
    - 添加的代码，查找的代码，都是比较简单的。 可以去尝试一下
    - 删除的代码有点难度。不用管
    - 扩展：B树，B+树，这些了解一下。优点，应用场景。 后期工作了之后可以花点时间学学
- 扩展一下
  - 数据库，索引，采用是B+树。 —— 什么是索引，就是为了找数据更快，不用每次找数据的时候，都遍历整张表 比如公司中，有一个表，1000w，现在需要在里面找 名字叫做 "zhangsan"， 快速找到一个你想找到的东西
  - B树：每一层的孩子节点都多， 二叉树，一个结点最多两个，B树可能一层有1000个孩子。 相同层高下，能装的数据会非常多
    - 为什么降低树的高度有好处，因为树是存储在磁盘中。 树层高高，读磁盘的次数会多，磁盘的速度非常慢，所以导致了数据库的速度非常慢
    - 如果对B树，四层能装多少数据？ 1000*1000*1000 = 10亿
    - 为什么不是用二叉树，二叉搜索树来存索引。 因为数据都是存储在磁盘中，磁盘的速度很慢。 10ms * 20 次 == 200ms = 0.2s 10ms * 2 次

哈希表

代码比较难。 建议大家理解一下代码，要写一下代码。
如果实在写不出来，重点应该是理解结构 + 添加流程。

- 结构和添加流程一定要记住
  - 1.简单说说你所认识的哈希表。
  - 存储键值对的数据结构。 底层结构是数组+链表。
  - 比如，现在要存储进去一个键值对。 1.使用哈希函数，对这个key进行计算，得到一个int。 2.使用得到的int，对数组长度进行取余。 之后其实这个键值对就像往这个下标存储 3.去该位置检查是否有数据。 如果没有数据，这时候直接将数据存进去。 如果这个位置有数据。 按个比较，键是否相同。 如果键不相同，这时候比较下一个； 键相同，用新值覆盖旧值；
  - 底层结构就是数组+链表
  - 常见的面试题基本都一个回答思路
    - 1.哈希表效率为啥高： 其实就是让你说底层结构 + 添加流程
    - 2.为啥哈希表底层结构是数组+链表
    - 3.哈希表底层结构是啥
    - 要听懂面试官的问题， 碰到自己会说的多说点。 往自己擅的上面聊
- 代码一定要掌握的
  - 添加的代码
  - 查找的代码
  - 删除
    - 如果能掌握就掌握。如果掌握不了，就先放一下。
    - 如果会了添加和查找， 对于删除的大致逻辑是没问题的。 但是对于它的指针挪动确实有点复杂
- 加载因子
  - 就是用来评判哈希表数据稀疏程度的。 键值对数目 / 数组长度。
  - 5/10 10/10 1000 / 10
  - 加载因子越大， trade off 能装更多的数据，但是效率受到了影响。 如果加载因子超过了一定的数值，
  - 实践过程中，一般采用的是一个阈值， 3/4
  - 这个主要是工程中采用的方式。 到达了合适的时机，就将数组变大点，保证效率。
- 对于传统的数组的添加、查找流程。代码一定要掌握。 如果删除能掌握更好
- 对于动态的数组，建议了解一下。 在面试的时候，被问到数据量太多的时候，效率低咋办。 知道有扩容这个动作在这里可以保证效率就可以了。